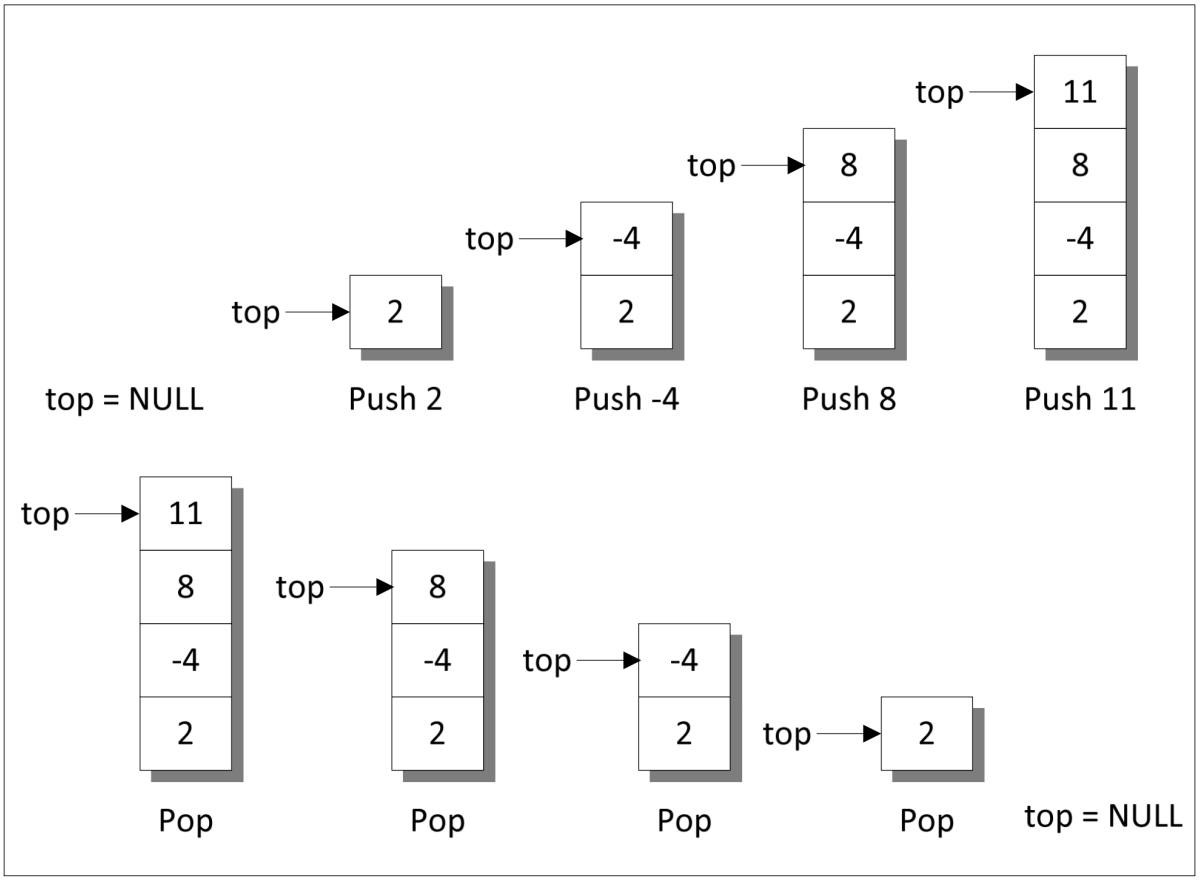
Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Unit 2 - Stacks

A stack is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle, meaning the last element added is the first one to be removed. All insertions and deletions occur at the same end, known as the **top** of the stack.

Imagine a stack of plates in a cafeteria. When a new plate is added, it is placed on top of the stack. Similarly, when a plate is removed, it is always taken from the top. This ensures that the most recently added plate is the first one to be taken away.

Stack has several important applications, including managing function calls through the call stack in recursion, evaluating expressions, handling undo and redo operations in editors, and implementing backtracking algorithms.



A stack data structure can be maintained as an array or as a linked list.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Stack Operations

Operation	Description
push(x)	Adds a new element (X) to the top of the stack.
pop()	Removes the topmost element from the stack.
peek()	Returns the top element from the stack.
isEmpty()	Checks if the stack is empty.
isFull()	Check if the stack is full.

Array Implementation

```
#define MAX 5
int stack[MAX], top = -1;

void push(int x) {
    if(top == MAX-1) printf("Stack Overflow!");
    else stack[++top] = x;
}

int pop() {
    if(top == -1) printf("Stack Underflow!");
    else return stack[top--];
}
```

Implementing Multiple Stacks in a Single Array

Two Stacks in One Array

- Approach 1 (Divide Space):
  - Stack 1 grows from left to right (0 to n/2).
  - Stack 2 grows from right to left (n/2+1 to n-1).
- Approach 2 (Efficient Space Utilization):
  - Stack 1 grows from start (0).
  - Stack 2 grows from end (MAX-1).
  - Overflow Condition: top1 + 1 == top2
  - Advantages:
    - Efficient memory usage (no fixed division).
    - No wastage of space until the entire array is full.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Algorithm

```
int arr[MAX], top1 = -1, top2 = MAX;

void push1(int x) {
    if(top1 + 1 == top2) printf("Stack Overflow!");
    else arr[++top1] = x;
}

void push2(int x) {
    if(top2 - 1 == top1) printf("Stack Overflow!");
    else arr[--top2] = x;
}
```

Linked List Implementation

A stack can be efficiently implemented using a singly linked list where:

- Push → Adds a node at the head
- Pop → Removes the node from the head

Node Structure

```
struct Node {
    int data;
    struct Node* next;
};
```

Stack Operations - Algorithms

1. Initialize Stack

```
struct Node* top = NULL; // Global stack pointer
```

2. Push Operation

1. Create a new node.
2. Set new_node->data = value.
3. Link the new node to the current top.
4. Update top to point to the new node.

3. Pop Operation

1. Check if the stack is empty (top == NULL).
2. Save top->data in a temporary variable.
3. Move top to top->next.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

4. Free the old top node.
5. Return the saved value.

Full C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int value) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct
Node));
    new_node->data = value;
    new_node->next = top;
    top = new_node;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow!\n");
        return -1;
    }
    struct Node* temp = top;
    int popped_value = top->data;
    top = top->next;
    free(temp);
    return popped_value;
}

int peek() {
    if (top == NULL) {
        printf("Stack is Empty!\n");
        return -1;
    }
    return top->data;
}
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

```c
int isEmpty() {
    return (top == NULL);
}

void display() {
    struct Node* temp = top;
    printf("Stack: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    push(10);
    push(20);
    push(30);
    display(); // Output: 30 -> 20 -> 10 -> NULL

    printf("Top element: %d\n", peek()); // 30

    pop();
    display(); // Output: 20 -> 10 -> NULL

    return 0;
}
```

Advantages of Linked List Implementation

- Dynamically resizes without a fixed capacity like arrays
- No overflow issues unless system memory is exhausted
- Push and pop operations are performed in constant O(1) time

Applications of Stacks

1. Function Call Management

Stacks are used to manage function calls in programming. When a function is called, its details (parameters, return address) are pushed onto the call stack. When the function returns, it is popped from the stack. This supports recursion and nested function calls.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

2. Undo/Redo Operations

Text editors (e.g., Notepad, Photoshop) use stacks to implement undo/redo:

- Undo: Pop the last action from the stack.
- Redo: Push the undone action back onto a secondary stack.

3. Balanced Parentheses Check

Compilers use stacks to validate syntax (e.g., matching (), {}, []).

Algorithm

1. Scan the expression left-to-right.
2. Push opening brackets (, {, [onto the stack.
3. When a closing bracket), },] is encountered:
4. Pop the stack and check if it matches.
5. If the stack is empty at the end, the expression is balanced.

4. Tower of Hanoi (Recursion Problem)

A classic recursion problem solved using stacks to represent pegs. The goal is to move disks from one peg to another, following rules:

- Only one disk can be moved at a time.
- A larger disk cannot be placed on a smaller one.

Stack Role:Each peg acts as a stack where disks are pushed and popped during moves.

5. Expression Conversion (Infix, Prefix, Postfix)

Stacks are often used in evaluation of arithmetic expressions. An arithmetic expression consists of operands and operators. The operands can be constant or variables. The operators used in an arithmetic expression can be +, -, * and /. A stack, manages operator precedence and helps in computing intermediate results. While writing an arithmetic expression, the operator is placed between two operands as shown in the examples below.

A + B * C
A * B - C
A + B / C - D
A $ B + C

This way of representing arithmetic expressions is called **infix notation**.

While evaluating an infix expression usually the following operator precedence is used:

- Highest priority: Exponentiation ($)
- Next highest priority: Multiplication (*) and Division (/)
- Lowest priority: Addition (+) and Subtraction (-)

If we wish to override these priorities we can do so by using a pair of parentheses as shown:

(A + B) * C
(A + B) / (C - D)

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

The expressions within a pair of parentheses are always evaluated earlier than other operations.

To make evaluation of an arithmetic expression easy, a Polish mathematician Jan Lukasiewicz suggested a notation called **Polish notation**. As per this notation, an expression in infix form can be converted to either prefix or postfix form and then evaluated. In prefix notation the operator comes before the operands. In postfix notation, the operator follows the two operands. These forms are shown below.

Infix form	-	A + B	Operator between operands
Prefix form	-	+ A B	Operator before operands
Postfix form	-	A B +	Operator after operands

The prefix and postfix expressions have three features:

- The operands maintain the same order as in the equivalent infix expression
- Parentheses are not needed to designate the expression unambiguously.
- While evaluating the expression the priority of the operators is irrelevant.

The stack data structure is used while carrying out the conversion of an expression given in one form to another. It is used in compilers and calculators for efficient expression handling.

Infix to Postfix Conversion (Shunting Yard Algorithm)

Algorithms

1. Initialize an empty stack and an empty output string.
2. Scan each character in the infix expression:
    a. If operand, add to output.
    b. If (, push to stack.
    c. If), pop and add to output until (is encountered.
    d. If operator, pop higher or equal precedence operators before pushing.
    e. Pop remaining operators from the stack.

Example

Convert A + B * C to Postfix

Symbol	**Stack**	**Output**	**Action**
A	-	A	Output A
+	+	A	Push +
B	+	A B	Output B
*	+ *	A B	Push * (higher precedence)
C	+ *	A B C	Output C
End	-	A B C * +	Pop all operators

Final Postfix Expression: A B C * +

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Convert (P + Q) * (R - S / T) to Postfix

Symbol	**Stack**	**Output**	**Action**
(	[( ]		Push (
P	[( ]	P	Output P
+	[(, +]	P	Push +
Q	[(, +]	P Q	Output Q
)	[ ]	P Q +	Pop + and output, discard (
*	[*]	P Q +	Push *
(	[*, ( ]	P Q +	Push (
R	[*, ( ]	P Q + R	Output R
-	[*, (, -]	P Q + R	Push -
S	[*, (, -]	P Q + R S	Output S
/	[*, (, -, /]	P Q + R S	Push /
T	[*, (, -, /]	P Q + R S T	Output T
)	[*]	P Q + R S T / -	Pop until (→ Output, discard (
End	[ ]	P Q + R S T / - *	Pop remaining *

Final Postfix Expression: P Q + R S T / - *

Infix to Prefix Conversion

Algorithm

1. Reverse the infix expression.
2. Convert to postfix.
    a. Initialize an empty stack and an empty output string.
    b. Scan each character in the infix expression:
        i. If operand, add to output.
        ii. If (, push to stack.
        iii. If), pop and add to output until (is encountered.
        iv. If operator, pop higher or equal precedence operators before pushing.
        v. Pop remaining operators from the stack.
3. Reverse the result.

Example

1. Convert (A + B) * C to Prefix

Step 1: Reverse → C * (B + A)
Step 2: Convert to Postfix:

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Symbol	Stack	Output	Action
C	-	C	Output C
*	*	C	Push *
(	* (	C	Push (
B	* (	C B	Output B
+	* (+	C B	Push +
A	* (+	C B A	Output A
)	*	C B A +	Pop until (
End	-	C B A + *	Pop all

Step 3: Reverse → * + A B C

Final Prefix Expression: * + A B C

2. Convert (P + Q) * (R - S / T) to Prefix

Step 1: Reverse and Swap Brackets

Note: while reversing each '(' will become ')' and each ')' becomes '('.

Original: (P + Q) * (R - S / T)

Reversed:) T / S - R (*) Q + P (

New infix to process after swapping brackets:

(T / S - R) * (Q + P)

We will convert this to postfix using a stack, then reverse the result to get a prefix.

Step 2: Convert to Postfix:

Symbol	Stack	Output	Action
(	(		Push (
T	(	T	Output T
/	(/	T	Push /
S	(/	T S	Output S
-	(-	T S /	Pop /, then push -
R	(-	T S / R	Output R
)		T S / R -	Pop till (
*	*	T S / R -	Push *
(	* (	T S / R -	Push (

Q	* (	T S / R - Q	Output Q
+	* (+	T S / R - Q	Push +
P	* (+	T S / R - Q P	Output P
)	*	T S / R - Q P +	Pop till (
End		T S / R - Q P + *	Pop remaining operator(s)

Postfix Expression: T S / R - Q P + *

Step 3: Reverse Postfix to get Prefix

Final Prefix Expression: * + P Q - R / S T

Postfix Evaluation

Algorithm

1. Initialize an empty stack.
2. Scan each character in the postfix expression:
    a. If operand, push to stack.
    b. If operator, pop two operands, apply operator, push result.
3. The final result is at the top of the stack.

Example

Evaluate 5 3 2 * +

Symbol	Stack	Action
5	[5]	Push 5
3	[5, 3]	Push 3
2	[5, 3, 2]	Push 2
*	[5, 6]	Pop 3 & 2, Calculate 3*2=6, Push 6
+	[11]	Pop 5 & 6, Calculate 5+6=11, Push 11

Result: 11

Practice Problem

Convert (A * B) + (C / D) to Postfix and Prefix using Stack.

- Postfix → A B * C D / +
- Prefix → + * A B / C D